

MMARTIN CLÍNICA VETERINARIA

Aplicación Web de Gestión Veterinaria

DOCUMENTACIÓN TÉCNICA DEL PROYECTO

Ciclo Formativo de Grado Superior – Desarrollo de Aplicaciones Web

Stack Tecnológico: Laravel 11 + React 19 + Vite + Tailwind CSS

Despliegue: Hostinger VPS · MySQL · SMTP

02/05/2026

1. Descripción y Objetivos Generales del Proyecto

1.1 Objetivos del Proyecto

El objetivo principal de este proyecto es desarrollar una aplicación web completa de gestión para la Clínica Veterinaria Mmartin, sustituyendo procesos manuales o sistemas dispersos por una plataforma unificada, accesible desde cualquier dispositivo con conexión a internet.

Los objetivos específicos del proyecto son:

- Desarrollar un sistema CRUD completo para la gestión de clientes (dueños de mascotas) y sus animales, incluyendo registro de fotos, datos clínicos y trazabilidad histórica.
- Implementar un módulo de historial médico digital que permita registrar diagnósticos, tratamientos, archivos adjuntos (radiografías, analíticas) y el descuento automático de medicación del inventario.
- Crear un calendario interactivo de citas que permita al personal sanitario agendar consultas y que los clientes puedan solicitar citas desde su portal personal.
- Integrar un sistema de notificaciones por correo electrónico para confirmaciones, cancelaciones y recordatorios de citas automáticos.
- Desarrollar un módulo de inventario con alertas de stock mínimo y consumo unitario de materiales durante la consulta.
- Implementar autenticación segura con soporte para Google OAuth y sistema de roles diferenciados (veterinario, asistente, cliente).
- Generar facturas en PDF automáticamente al finalizar una consulta médica.
- Desplegar la aplicación en un entorno de producción real con dominio propio y base de datos en la nube.

1.2 Motivación

La motivación nace de la necesidad real de modernizar la gestión de pequeñas clínicas veterinarias, que en muchos casos siguen utilizando libros de registro físicos, hojas de cálculo o programas de escritorio obsoletos. Estas herramientas dificultan el acceso a la información histórica de los pacientes, no ofrecen notificaciones automáticas a los clientes y no permiten el acceso remoto al personal.

Con la proliferación de smartphones y la expectativa de los usuarios de interactuar con los servicios de su mascota de forma digital, existe una oportunidad clara de aportar valor tecnológico a este sector. El proyecto también representa una oportunidad de aprendizaje completo del ciclo de vida de una aplicación web moderna: desde el diseño de la base de datos hasta el despliegue en producción, pasando por el desarrollo de una API RESTful y una SPA (Single Page Application) con React.

Además, el proyecto incluye elementos avanzados como la generación de documentos PDF, la integración de OAuth con Google y el envío de correos transaccionales, que son competencias muy demandadas en el mercado laboral actual del desarrollo web.

1.3 Descripción Funcional de la Aplicación

La aplicación web Mmartin Clínica Veterinaria es una SPA (Single Page Application) de tipo full-stack que opera en modo cliente-servidor. El backend expone una API RESTful desarrollada en Laravel 11 y el frontend es una interfaz desarrollada en React 19 compilada con Vite.

La aplicación distingue tres tipos de usuarios con acceso diferenciado:

- Veterinario: acceso completo a todas las funcionalidades del sistema, incluyendo gestión de personal.
- Asistente: acceso a la gestión de clientes, mascotas, citas e inventario, sin gestión de usuarios.
- Cliente: acceso a un portal personal para consultar el historial médico de sus mascotas y solicitar citas.

Los módulos funcionales principales de la aplicación son:

- Dashboard: panel de control con métricas en tiempo real (total de pacientes, clientes, citas del día, alertas de stock).
- Gestión de Clientes: alta, edición y búsqueda de propietarios con su ficha de contacto completa.
- Gestión de Mascotas: registro de animales vinculados a su propietario, con foto, especie, raza, peso y fecha de nacimiento.
- Historial Médico: registro de consultas con diagnóstico, tratamiento, adjuntos y medicación aplicada.
- Calendario de Citas: visualización mensual, semanal y diaria de citas con creación, edición y eliminación.
- Inventario: gestión del stock de medicamentos y materiales con alertas y consumo rápido.
- Facturación: generación automática de facturas en PDF por consulta con desglose de IVA.
- Correos electrónicos: notificaciones automáticas de confirmación, recordatorio y cancelación de citas.
- Gestión de Personal: alta de usuarios empleados con asignación de rol.
- Portal del Cliente: acceso restringido para dueños de mascotas con solicitud de citas online.

1.4 Características y Entorno de Desarrollo

El entorno de desarrollo ha sido configurado con las siguientes herramientas y tecnologías:

Categoría	Tecnología / Herramienta	Versión
Backend Framework	Laravel	11.x
Lenguaje Backend	PHP	8.2+
Frontend Framework	React	19.x
Bundler Frontend	Vite	6.x
CSS Framework	Tailwind CSS	3.x (CDN)
Base de Datos	MySQL	8.0
ORM	Eloquent (Laravel)	Integrado
Autenticación	Laravel Sanctum + Google OAuth	Integrado
Generación PDF	DomPDF (Blade)	Integrado
Correo SMTP	Mailtrap (dev) / SMTP real (prod)	—
Calendario UI	react-big-calendar	1.x
Iconos	Font Awesome 7 (React)	7.x

Alertas UI	SweetAlert2	11.x
HTTP Client	Axios	1.x
IDE	Visual Studio Code	—
Control de Versiones	Git + GitHub	—
Servidor Local	php artisan serve / Vite dev	—

1.5 Elementos de Investigación

Durante el desarrollo del proyecto se han investigado y aplicado los siguientes elementos técnicos que requirieron estudio previo:

- Integración de Google OAuth 2.0 con Laravel Sanctum: implementación del flujo de autenticación OAuth con redirección, obtención del token de Google y generación del token de API de la aplicación.
- Generación de PDF con DomPDF en Laravel: creación de vistas Blade específicas para facturas con estilos CSS inline compatibles con el motor de renderizado de DomPDF.
- Envío de correos con plantillas Blade y la clase Mail de Laravel: diseño de mailable classes y vistas de correo con el componente x-mail::message.
- Gestión de archivos adjuntos: subida de imágenes y documentos PDF al sistema de almacenamiento de Laravel con acceso público mediante links simbólicos.
- Calendario interactivo con react-big-calendar: localización en español, slots de selección, eventos con datos enriquecidos y personalización del componente de evento.
- Sistema de roles y guards en Laravel Sanctum: protección de rutas según el rol del usuario autenticado.
- Despliegue en Hostinger con acceso SSH: configuración del servidor Apache, variables de entorno de producción, migración de base de datos remota y enlace de almacenamiento.

2. Planificación del Proyecto

2.1 Análisis de Requisitos Funcionales

Los requisitos funcionales describen las capacidades que el sistema debe ofrecer a sus usuarios. Se han identificado mediante entrevistas con el cliente final y análisis de flujos de trabajo de la clínica.

ID	Requisito Funcional	Prioridad
RF-01	El sistema debe permitir registrar propietarios con nombre, email, teléfono y dirección.	Alta
RF-02	El sistema debe permitir registrar mascotas asociadas a un propietario, incluyendo foto.	Alta
RF-03	El sistema debe gestionar el historial médico por mascota con diagnóstico y tratamiento.	Alta
RF-04	El sistema debe permitir adjuntar archivos (PDF, imágenes) a registros médicos.	Media
RF-05	El sistema debe descontar automáticamente del inventario el producto usado en consulta.	Alta
RF-06	El sistema debe generar una factura en PDF al finalizar una consulta.	Alta
RF-07	El sistema debe permitir crear, editar y eliminar citas en un calendario interactivo.	Alta
RF-08	El sistema debe enviar correo de confirmación al agendar una cita.	Alta
RF-09	El sistema debe enviar recordatorio automático 24 horas antes de la cita.	Media
RF-10	El sistema debe enviar correo de cancelación si se elimina una cita.	Media
RF-11	El sistema debe gestionar el inventario con alertas de stock mínimo.	Alta
RF-12	Los clientes deben acceder a un portal personal con su historial.	Alta
RF-13	Los clientes deben poder solicitar citas desde el portal.	Media
RF-14	El sistema debe soportar inicio de sesión con cuenta Google.	Media
RF-15	El veterinario debe poder gestionar los usuarios del sistema con roles.	Alta

2.2 Análisis de Requisitos No Funcionales

Los requisitos no funcionales definen las restricciones de calidad, rendimiento y seguridad del sistema:

ID	Requisito No Funcional	Categoría
RNF-01	La aplicación debe ser accesible desde cualquier navegador moderno sin instalación.	Usabilidad
RNF-02	La interfaz debe ser responsive y funcionar correctamente en dispositivos móviles.	Usabilidad
RNF-03	Las contraseñas deben almacenarse con hash bcrypt (12 rondas).	Seguridad
RNF-04	Las rutas de la API deben estar protegidas con tokens de autenticación (Sanctum).	Seguridad
RNF-05	Las comunicaciones deben realizarse sobre HTTPS en producción.	Seguridad
RNF-06	El tiempo de respuesta de la API no debe superar los 2 segundos en condiciones normales.	Rendimiento
RNF-07	El sistema debe ser capaz de gestionar al menos 500 clientes y 1000 mascotas.	Escalabilidad
RNF-08	La base de datos debe tener copias de seguridad automáticas diarias.	Disponibilidad
RNF-09	El código fuente debe estar versionado en Git con commits descriptivos.	Mantenibilidad

2.3 Modelo de Negocio

La aplicación está orientada a clínicas veterinarias de pequeño y mediano tamaño que necesitan digitalizar su gestión interna. El modelo de negocio se basa en el despliegue del software como producto a medida para el cliente final, con la posibilidad de evolucionar a un modelo SaaS (Software as a Service) con suscripción mensual para múltiples clínicas.

Los actores del modelo de negocio son:

- Clínica Veterinaria (cliente): contrata el software para gestionar su operativa diaria. Se beneficia de la reducción de tiempo en tareas administrativas, la digitalización de expedientes clínicos y la mejora en la comunicación con sus clientes.
- Propietarios de mascotas (usuarios finales): acceden al portal del cliente para consultar el estado de salud de su mascota y gestionar citas de forma autónoma.
- Personal de la clínica (veterinarios y asistentes): utilizan el sistema para el trabajo diario, registrando consultas, gestionando el stock y administrando el calendario.

El flujo de valor principal es: el propietario solicita cita → el sistema notifica al veterinario → se realiza la consulta → el sistema registra el historial médico y genera la factura → el propietario recibe el resumen por correo.

3. Diseño de la Base de Datos

3.1 Diagrama de Clases y Modelo de Datos

La base de datos está diseñada en MySQL y gestionada mediante las migraciones de Laravel (Eloquent). A continuación se describe cada tabla del modelo relacional:

Tabla: users

Almacena todos los usuarios del sistema, tanto personal de la clínica como propietarios de mascotas registrados.

Campo	Tipo	Restricciones	Descripción
id	BIGINT UNSIGNED	PK, AUTO_INCREMENT	Identificador único
name	VARCHAR(255)	NOT NULL	Nombre completo del usuario
email	VARCHAR(255)	NOT NULL, UNIQUE	Correo electrónico (login)
password	VARCHAR(255)	NULLABLE	Hash bcrypt (null si OAuth)
role	ENUM	NOT NULL	veterinario / asistente / client
google_id	VARCHAR(255)	NULLABLE, UNIQUE	ID de Google OAuth
owner_id	BIGINT UNSIGNED	NULLABLE, FK	Referencia al propietario asociado
created_at / updated_at	TIMESTAMP	—	Marcas temporales automáticas

Tabla: owners

Almacena los datos de los propietarios de mascotas como clientes de la clínica.

Campo	Tipo	Restricciones	Descripción
id	BIGINT UNSIGNED	PK, AUTO_INCREMENT	Identificador único
name	VARCHAR(255)	NOT NULL	Nombre del propietario
email	VARCHAR(255)	NOT NULL, UNIQUE	Email de contacto
telefono	VARCHAR(20)	NULLABLE	Teléfono de contacto
direccion	VARCHAR(255)	NULLABLE	Dirección postal
created_at / updated_at	TIMESTAMP	—	Marcas temporales

Tabla: pets

Almacena los datos de las mascotas, vinculadas a su propietario.

Campo	Tipo	Restricciones	Descripción
id	BIGINT UNSIGNED	PK, AUTO_INCREMENT	Identificador único
owner_id	BIGINT UNSIGNED	FK (owners.id), NOT NULL	Propietario de la mascota
name	VARCHAR(100)	NOT NULL	Nombre de la mascota
species	VARCHAR(50)	NOT NULL	Especie (Perro, Gato, Ave...)
raza	VARCHAR(100)	NULLABLE	Raza del animal
peso	DECIMAL(5,2)	NULLABLE	Peso en kilogramos
fecha_nac	DATE	NULLABLE	Fecha de nacimiento
photo_path	VARCHAR(255)	NULLABLE	Ruta a la foto en storage

Tabla: appointments

Almacena las citas de las mascotas, con su estado y tipo de consulta.

Campo	Tipo	Restricciones	Descripción
id	BIGINT UNSIGNED	PK, AUTO_INCREMENT	Identificador único
owner_id	BIGINT UNSIGNED	FK (owners.id)	Propietario que solicita la cita
pet_id	BIGINT UNSIGNED	FK (pets.id)	Mascota para la que se cita
title	VARCHAR(255)	NOT NULL	Motivo / título de la consulta
start_time	DATETIME	NOT NULL	Fecha y hora de la cita
notes	TEXT	NULLABLE	Notas adicionales
type	VARCHAR(50)	DEFAULT 'consultation'	Tipo: consultation, vaccine...
status	VARCHAR(50)	DEFAULT 'scheduled'	Estado: scheduled, pending, cancelled

Tabla: medical_records

Almacena el historial médico de cada mascota, con posibilidad de adjuntar archivos.

Campo	Tipo	Restricciones	Descripción
id	BIGINT UNSIGNED	PK, AUTO_INCREMENT	Identificador único
pet_id	BIGINT UNSIGNED	FK (pets.id), NOT NULL	Mascota del registro
symptom_title	VARCHAR(255)	NOT NULL	Motivo de la consulta / síntoma
diagnosis	TEXT	NULLABLE	Diagnóstico del veterinario
treatment	TEXT	NULLABLE	Tratamiento prescrito
attachment_path	VARCHAR(255)	NULLABLE	Ruta del archivo adjunto
attachment_type	VARCHAR(50)	NULLABLE	Tipo MIME del adjunto
product_id	BIGINT UNSIGNED	NULLABLE, FK (products.id)	Medicamento dispensado

Tabla: products

Almacena el inventario de medicamentos y materiales de la clínica.

Campo	Tipo	Restricciones	Descripción
id	BIGINT UNSIGNED	PK, AUTO_INCREMENT	Identificador único
name	VARCHAR(255)	NOT NULL	Nombre del producto
description	TEXT	NULLABLE	Descripción del producto
category	VARCHAR(100)	NULLABLE	Categoría (vacuna, antibiótico...)
price	DECIMAL(8,2)	NOT NULL	Precio unitario
stock_quantity	INT	NOT NULL, DEFAULT 0	Unidades en stock
min_stock_alert	INT	NOT NULL, DEFAULT 5	Umbral de alerta de stock bajo

Las relaciones entre tablas son:

- owners tiene muchas pets (1:N)
- pets pertenece a un owner y tiene muchos medical_records y appointments (1:N)
- appointments pertenece a un owner y a un pet
- medical_records pertenece a un pet y puede referenciar a un product
- users puede estar vinculado opcionalmente a un owner (para el rol cliente)

4. Documentación de la Aplicación

4.1 Instalación

A continuación se describe el proceso de instalación del proyecto en un entorno local de desarrollo.

Requisitos previos

- PHP 8.2 o superior con extensiones: pdo_mysql, mbstring, openssl, fileinfo, gd
- Composer 2.x (gestor de dependencias PHP)
- Node.js 20.x o superior con npm
- MySQL 8.0 o superior
- Git para el control de versiones

Clonación del repositorio

```
git clone https://github.com/Miguelmntz/Veterinarian
cd Veterinarian
```

Instalación del backend (Laravel)

```
cd back
composer install
cp .env.example .env
php artisan key:generate
```

Editar el archivo .env con los parámetros de la base de datos local:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=veterinaria_local
DB_USERNAME=root
DB_PASSWORD=tu_password
```

Instalación del frontend (React + Vite)

```
cd ../front # directorio del frontend
npm install
```

Enlace de almacenamiento

```
php artisan storage:link
```

Este comando crea un enlace simbólico desde public/storage hacia storage/app/public, necesario para acceder a las imágenes subidas.

4.2 Creación de Recursos Principales

4.2.1 Modelos, Controladores, Migraciones, Factories y Seeders

Laravel proporciona el comando artisan make:model con opciones para generar todos los archivos relacionados en una sola instrucción. A continuación se muestran los comandos utilizados para crear los recursos principales del proyecto:

```
# Crear modelo con migración, factory, seeder y controlador de recurso
php artisan make:model Owner -mfsc
```

```

php artisan make:model Pet -mfsc
php artisan make:model Appointment -mfsc
php artisan make:model MedicalRecord -mfsc
php artisan make:model Product -mfsc

# Crear controladores API adicionales
php artisan make:controller Auth/LoginController
php artisan make:controller Auth/GoogleController
php artisan make:controller InvoiceController

# Crear clases de correo (Mailables)
php artisan make:mail AppointmentConfirmed
php artisan make:mail AppointmentCancelled
php artisan make:mail AppointmentReminder
php artisan make:mail MedicalRecordVisitSummary

# Crear comando para recordatorios automáticos
php artisan make:command SendAppointmentReminders

```

4.2.2 Migraciones

Las migraciones de Laravel permiten versionar el esquema de la base de datos. A continuación se muestran los aspectos más relevantes de cada migración:

Migración: create_owners_table

```

public function up(): void
{
    Schema::create('owners', function (Blueprint $table) {
        $table->id();
        $table->string('name');
        $table->string('email')->unique();
        $table->string('telefono')->nullable();
        $table->string('direccion')->nullable();
        $table->timestamps();
    });
}

```

Migración: create_pets_table

```

public function up(): void
{
    Schema::create('pets', function (Blueprint $table) {
        $table->id();
        $table->foreignId('owner_id')->constrained()->onDelete('cascade');
        $table->string('name', 100);
        $table->string('species', 50);
        $table->string('raza', 100)->nullable();
        $table->decimal('peso', 5, 2)->nullable();
    });
}

```

```

        $table->date('fecha_nac')->nullable();
        $table->string('photo_path')->nullable();
        $table->timestamps();
    });
}

```

Migración: create_appointments_table

```

Schema::create('appointments', function (Blueprint $table) {
    $table->id();
    $table->foreignId('owner_id')->constrained('owners')->onDelete('cascade');
    $table->foreignId('pet_id')->constrained('pets')->onDelete('cascade');
    $table->string('title');
    $table->dateTime('start_time');
    $table->text('notes')->nullable();
    $table->string('type', 50)->default('consultation');
    $table->string('status', 50)->default('scheduled');
    $table->timestamps();
});

```

Migración: create_medical_records_table

```

Schema::create('medical_records', function (Blueprint $table) {
    $table->id();
    $table->foreignId('pet_id')->constrained()->onDelete('cascade');
    $table->string('symptom_title');
    $table->text('diagnosis')->nullable();
    $table->text('treatment')->nullable();
    $table->string('attachment_path')->nullable();
    $table->string('attachment_type', 50)->nullable();
    $table->foreignId('product_id')->nullable()->constrained()->nullOnDelete();
    $table->timestamps();
});

```

Migración: create_products_table

```

Schema::create('products', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->text('description')->nullable();
    $table->string('category', 100)->nullable();
    $table->decimal('price', 8, 2);
    $table->integer('stock_quantity')->default(0);
    $table->integer('min_stock_alert')->default(5);
    $table->timestamps();
});

```

4.2.3 Factories

Las factories de Laravel utilizan la librería Faker para generar datos ficticios con aspecto realista. Se configuró el locale es_ES para que los datos estén en español.

```
# En .env
APP_FAKER_LOCALE=es_ES
```

Ejemplo de la OwnerFactory:

```
class OwnerFactory extends Factory
{
    public function definition(): array
    {
        return [
            'name'      => fake()->name(),
            'email'      => fake()->unique()->safeEmail(),
            'telefono'   => fake()->phoneNumber(),
            'direccion' => fake()->address(),
        ];
    }
}
```

Ejemplo de la PetFactory:

```
class PetFactory extends Factory
{
    public function definition(): array
    {
        $species = fake()->randomElement(['Perro', 'Gato', 'Ave',
        'Conejo']);
        $breeds  = ['Perro' => ['Labrador', 'Beagle', 'Bulldog', 'Mestizo'],
                    'Gato'  => ['Europeo', 'Persa', 'Siamés', 'Maine
        Coon']];
        return [
            'owner_id' => Owner::factory(),
            'name'      => fake()->firstName(),
            'species'   => $species,
            'raza'      => $breeds[$species][array_rand($breeds[$species])]
            ?? null,
            'peso'      => fake()->randomFloat(1, 1, 40),
            'fech_nac'  => fake()->dateTimeBetween('-15 years', '-1 month'),
        ];
    }
}
```

4.2.4 Modelos

Los modelos de Eloquent definen las relaciones entre las tablas y los atributos fillable para asignación masiva. A continuación se muestran los modelos principales:

Modelo Owner

```
class Owner extends Model
{

```

```

        protected $fillable = ['name', 'email', 'telefono', 'direccion'];

        public function pets(): HasMany
        { return $this->hasMany(Pet::class); }

        public function appointments(): HasMany
        { return $this->hasMany(Appointment::class); }
    }

```

Modelo Pet

```

class Pet extends Model
{
    protected $fillable =
    ['owner_id', 'name', 'species', 'raza', 'peso', 'fecha_nac', 'photo_path'];

    public function owner(): BelongsTo
    { return $this->belongsTo(Owner::class); }

    public function medicalRecords(): HasMany
    { return $this->hasMany(MedicalRecord::class); }
}

```

Modelo Appointment

```

class Appointment extends Model
{
    protected $fillable =
    ['owner_id', 'pet_id', 'title', 'start_time', 'notes', 'type', 'status'];

    public function owner(): BelongsTo
    { return $this->belongsTo(Owner::class); }

    public function pet(): BelongsTo
    { return $this->belongsTo(Pet::class); }
}

```

Modelo Product

```

class Product extends Model
{
    protected $fillable =
    ['name', 'description', 'category', 'price', 'stock_quantity', 'min_stock_alert'];

    // Scope para productos con stock bajo
    public function scopeLowStock($query)
    {
        return $query->whereColumn('stock_quantity', '<=',
        'min_stock_alert');
    }
}

```

```
    }
}
```

4.3 Uso de Recursos

4.3.1 Creación de Tablas

Para crear las tablas en la base de datos a partir de las migraciones, se utiliza el siguiente comando de Artisan:

```
php artisan migrate
```

Si es necesario revertir todas las migraciones y volver a ejecutarlas (útil durante el desarrollo):

```
php artisan migrate:fresh
```

4.3.2 Crear Datos Ficticios

Para poblar la base de datos con datos de prueba generados por las factories, se utiliza el comando:

```
php artisan db:seed
```

El DatabaseSeeder principal encadena los seeders individuales:

```
public function run(): void
{
    $this->call([
        UserRoleSeeder::class,    // Crea usuarios con diferentes roles
        OwnerSeeder::class,       // Crea 30 propietarios ficticios
        PetSeeder::class,         // Crea 2-4 mascotas por propietario
        ProductSeeder::class,     // Crea 20 medicamentos y materiales
        AppointmentSeeder::class, // Crea citas de las últimas semanas
    ]);
}
```

4.3.3 Crear Tablas y Datos en un Solo Paso

Durante el desarrollo y para las pruebas de integración, se utiliza el comando que combina ambas operaciones:

```
php artisan migrate:fresh --seed
```

Este comando elimina todas las tablas, las vuelve a crear y ejecuta todos los seeders, dejando la base de datos en un estado limpio y poblado.

4.4 Configuración de Controladores

Los controladores de la API siguen el patrón RESTful. Se utiliza la clase base Controller de Laravel y se retornan respuestas JSON estandarizadas. A continuación se documenta la lógica principal de los controladores más relevantes:

OwnerController

Gestiona el CRUD completo de propietarios. Carga las mascotas relacionadas en cada consulta para evitar múltiples peticiones desde el frontend.

```
public function index(): JsonResponse
{
    return response()->json(
```

```

        Owner::with('pets')->orderBy('name')->get()
    );
}

public function store(Request $request): JsonResponse
{
    $validated = $request->validate([
        'name'          => 'required|string|max:255',
        'email'          => 'required|email|unique:owners,email',
        'telefono'       => 'nullable|string',
        'direccion'      => 'nullable|string',
        'pets'           => 'nullable|array',
    ]);
    $owner = Owner::create($validated);
    if (!empty($validated['pets'])) {
        foreach ($validated['pets'] as $petData) {
            $owner->pets()->create($petData);
        }
    }
    return response()->json($owner->load('pets'), 201);
}

```

AppointmentController

Gestiona las citas. Al crear o cancelar, dispara el envío de correos automáticamente mediante la fachada Mail de Laravel.

```

public function store(Request $request): JsonResponse
{
    $appointment = Appointment::create($request->validated());
    $appointment->load(['owner', 'pet']);
    // Envío de correo de confirmación
    Mail::to($appointment->owner->email)
        ->send(new AppointmentConfirmed($appointment));
    return response()->json($appointment->load('owner','pet'), 201);
}

public function destroy(Appointment $appointment): JsonResponse
{
    $appointment->load(['owner', 'pet']);
    Mail::to($appointment->owner->email)
        ->send(new AppointmentCancelled($appointment));
    $appointment->delete();
    return response()->json(['message' => 'Cita eliminada'], 200);
}

```


MedicalRecordController

Gestiona los registros médicos con soporte para subida de archivos adjuntos y descuento automático de inventario.

```
public function store(Request $request): JsonResponse
{
    $data = $request->validate([...]);
    if ($request->hasFile('attachment')) {
        $path = $request->file('attachment')->store('medical_attachments',
'public');
        $data['attachment_path'] = $path;
        $data['attachment_type'] = $request->file('attachment')->getMimeType();
    }
    // Descuento de inventario
    if (!empty($data['product_id'])) {
        Product::find($data['product_id'])->decrement('stock_quantity');
    }
    $record = MedicalRecord::create($data);
    // Envío de resumen por correo al propietario
    Mail::to($record->pet->owner->email)
        ->send(new MedicalRecordVisitSummary($record));
    return response()->json($record->load('product'), 201);
}
```

InvoiceController

Genera facturas en PDF usando DomPDF con una vista Blade específica, incluyendo los datos del propietario, la mascota, el diagnóstico y el desglose de IVA al 21%.

```
public function download(MedicalRecord $record)
{
    $record->load('pet.owner', 'product');
    $consultationFee = 35.00;
    $productCost = $record->product?->price ?? 0;
    $totalCost = $consultationFee + $productCost;
    $iva = $totalCost * 0.21;
    $grandTotal = $totalCost + $iva;
    $pdf = Pdf::loadView('pdf.invoice', [
        'record'          => $record,
        'owner'           => $record->pet->owner,
        'pet'             => $record->pet,
        'product'         => $record->product,
        'consultationFee' => $consultationFee,
        'totalCost'       => $totalCost,
        'iva'             => $iva,
        'grandTotal'      => $grandTotal,
        'invoice_number' => 'FAC-' . str_pad($record->id, 5, '0',
STR_PAD_LEFT),
        'date'           => now()->format('d/m/Y'),
    ])
```

```

    });
    return $pdf->download('factura-'. $record->id.' .pdf');
}

```

Comando: SendAppointmentReminders

Comando Artisan programado para ejecutarse diariamente a las 9:00 AM que envía recordatorios de citas del día siguiente:

```

protected $signature = 'app:send-reminders';

public function handle(): void
{
    $tomorrow = Carbon::tomorrow();
    $appointments = Appointment::whereDate('start_time', $tomorrow)
        ->where('status', 'scheduled')
        ->with(['owner', 'pet'])->get();
    foreach ($appointments as $appointment) {
        Mail::to($appointment->owner->email)
            ->send(new AppointmentReminder($appointment));
    }
    $this->info("Enviados {$appointments->count()} recordatorios.");
}

```

4.5 Archivo de Rutas

Las rutas de la API se definen en routes/api.php. Se utilizan middlewares de autenticación de Sanctum y grupos de rutas para organizar el acceso por roles.

```

// Rutas públicas (sin autenticación)
Route::post('/login', [LoginController::class, 'login']);
Route::post('/register', [LoginController::class, 'register']);
Route::get('/auth/google', [GoogleController::class, 'redirect']);
Route::get('/auth/google/callback', [GoogleController::class, 'callback']);

// Rutas protegidas por token Sanctum
Route::middleware('auth:sanctum')->group(function () {
    Route::get('/user', fn(Request $r) => $r->user());
    Route::post('/logout', [LoginController::class, 'logout']);

    // Recursos CRUD principales
    Route::apiResource('owners', OwnerController::class);
    Route::apiResource('pets', PetController::class);
    Route::apiResource('appointments', AppointmentController::class);
    Route::apiResource('medical-records', MedicalRecordController::class);
    Route::apiResource('products', ProductController::class);

    // Rutas especiales
    Route::post('products/{product}/consume', [ProductController::class, 'consume']);
}

```

```

    Route::get('invoices/{record}/download', [InvoiceController::class,
'download']);

    Route::get('dashboard', [DashboardController::class,
'index']);

    // Gestión de usuarios (solo veterinario)
    Route::middleware('role:veterinario')->group(function () {
        Route::get('staff', [UserController::class, 'index']);
        Route::post('staff', [UserController::class, 'store']);
        Route::delete('staff/{user}', [UserController::class, 'destroy']);
    });
});

```

4.6 Búsquedas

La aplicación implementa búsquedas en tiempo real en el frontend mediante filtrado local del array de datos ya cargados. Para la gestión de clientes, el campo de búsqueda filtra por nombre, email y teléfono simultáneamente:

```

// Filtro en el componente React de Clientes
const filtered = owners.filter(o =>
    o.name.toLowerCase().includes(search.toLowerCase()) ||
    o.email?.toLowerCase().includes(search.toLowerCase()) ||
    o.telefono?.includes(search)
);

```

En el módulo de inventario, el filtrado contempla también la categoría del producto:

```

const filtered = products.filter(p =>
    p.name.toLowerCase().includes(search.toLowerCase()) ||
    p.category?.toLowerCase().includes(search.toLowerCase())
);

```

Para el modal de gestión de citas, la búsqueda de propietarios se realiza también localmente sobre el array ya cargado, mostrando resultados en un dropdown emergente a medida que el usuario escribe el nombre o teléfono.

5. Manual de Usuario

5.1 Página Principal (Dashboard)

El dashboard es la pantalla de inicio para el personal de la clínica (veterinario y asistente). Se accede automáticamente tras el inicio de sesión exitoso. La pantalla está dividida en varias zonas:

Alertas de citas pendientes

En la parte superior aparece un aviso destacado si existen citas con estado 'pendiente' (solicitadas por clientes a través del portal pero aún no confirmadas). Incluye el número de citas pendientes y un botón de acceso rápido al calendario para gestionarlas.

Tarjetas de métricas

Cuatro tarjetas resumen muestran en tiempo real:

- Total de dueños registrados en el sistema.
- Total de pacientes (mascotas) registradas.
- Número de citas programadas para el día actual.
- Número de alertas de stock bajo en el inventario (productos por debajo del umbral mínimo).

Cada tarjeta actúa como enlace de navegación rápida al módulo correspondiente.

Agenda del día

Lista todas las citas programadas para la jornada actual, mostrando la hora, el nombre de la mascota y el motivo. Si no hay citas, aparece un mensaje informativo.

Alertas de stock bajo

Lista los productos del inventario cuyo stock ha llegado o superado el umbral de alerta mínimo, con el stock actual resaltado en rojo para facilitar la reposición.

5.2 Login y Register

Inicio de Sesión con Email y Contraseña

El formulario de acceso solicita el correo electrónico y la contraseña. Las credenciales de prueba configuradas en el sistema son:

- Veterinario: admin@veterinario.com / password123
- Clientes: usan su email real y la contraseña por defecto password123

Si las credenciales son incorrectas, el sistema muestra un mensaje de error mediante SweetAlert2 sin revelar qué campo es incorrecto.

Inicio de Sesión con Google

El botón 'Continuar con Google' redirige al flujo de OAuth de Google. Una vez autenticado, Google devuelve el token al endpoint de callback, que busca o crea el usuario en la base de datos y genera un token de Sanctum para las siguientes peticiones a la API.

Portal del Cliente

Los usuarios con rol 'client' son redirigidos automáticamente al portal del cliente, una interfaz simplificada que muestra únicamente sus mascotas y las opciones de ver historial médico y solicitar cita.

5.3 Módulos

5.3.1 Módulo: Clientes y Mascotas

Accesible desde la barra de navegación (icono de personas). Este módulo muestra la lista de todos los propietarios registrados en el sistema con sus datos de contacto y mascotas asociadas.

Funcionalidades principales:

- Búsqueda en tiempo real por nombre, email o teléfono del propietario.
- Alta de nuevo cliente: formulario con nombre, email, teléfono, dirección y la posibilidad de añadir mascotas directamente en el mismo formulario.
- Edición del cliente: nombre, teléfono y dirección (el email es inmutable una vez registrado).
- Gestión de mascotas: desde la tarjeta de cada cliente se pueden añadir nuevas mascotas, editar las existentes (nombre, especie, raza, peso, fecha de nacimiento, foto) y eliminarlas.
- Acceso rápido al historial médico: el botón 'Ver Historial' abre el panel clínico de cada mascota.

5.3.2 Módulo: Historial Médico

Se accede desde la tarjeta de cada mascota pulsando 'Ver Historial'. Muestra un panel lateral dividido en dos columnas:

Columna izquierda – Historial existente:

- Lista cronológica de todas las consultas médicas de la mascota.
- Para cada consulta: fecha, motivo, diagnóstico, tratamiento, medicamento dispensado y archivo adjunto.
- Botones de descarga de la factura PDF y del archivo adjunto (radiografía o analítica).
- Botón de eliminación del registro (solo para veterinario).

Columna derecha – Redactar nueva consulta:

- Campo de motivo de consulta (obligatorio).
- Campos de diagnóstico y tratamiento (opcionales).
- Selector de medicamento del inventario (descuenta stock automáticamente al guardar).
- Campo para adjuntar un archivo (PDF o imagen).
- Al guardar, el sistema envía un email de resumen de visita al propietario.

5.3.3 Módulo: Inventario

Accesible desde la barra de navegación (icono de caja). Muestra todos los productos y medicamentos del inventario en forma de tabla.

Columnas de la tabla:

- Nombre del producto con badge de categoría.
- Stock actual con indicador de color (verde: normal, rojo: por debajo del umbral).
- Stock mínimo de alerta configurado.
- Precio unitario.
- Botón de consumo rápido: descuenta 1 unidad sin confirmación adicional.
- Botones de edición (lápiz) y eliminación (papelera).

El formulario de alta/edición de producto permite configurar: nombre, descripción, categoría, precio, stock actual y umbral de alerta mínimo.

5.3.4 Módulo: Calendario de Citas

Accesible desde la barra de navegación (icono de calendario). Muestra un calendario interactivo construido con react-big-calendar, localizado en español.

Vistas disponibles:

- Vista mensual: visión general con las citas del mes.
- Vista semanal: detalle de la semana con franjas horarias.
- Vista diaria: agenda de un día con horarios detallados.
- Vista agenda: lista textual de próximas citas.

Creación de cita:

Al pulsar sobre un slot del calendario se abre un modal de creación. El formulario incluye:

- Buscador de propietario con autocompletado (busca por nombre o teléfono).
- Selector de mascota del propietario elegido.
- Motivo de la cita.
- Fecha (preseleccionada con el slot pulsado).
- Hora de inicio con selector de intervalos de 15 minutos.
- Las horas ya ocupadas se muestran deshabilitadas y marcadas.
- Tipo de consulta y estado inicial.

Al guardar, el sistema envía automáticamente un correo de confirmación al propietario. Las citas pendientes (solicitadas por clientes) aparecen en color naranja; las confirmadas en azul índigo.

5.3.5 Módulo: Personal (Solo Veterinario)

Accesible desde el icono de escudo en la barra de navegación, visible únicamente para el rol 'veterinario'. Muestra la lista de empleados del sistema con su nombre, email y rol asignado.

Funcionalidades:

- Alta de nuevo empleado: nombre, email, contraseña temporal y rol (veterinario o asistente).
- Eliminación de empleado: revoca el acceso al sistema del usuario seleccionado.

Los usuarios con rol 'asistente' tienen acceso a todos los módulos excepto la gestión de personal.

5.3.6 Módulo: Perfil del Cliente

Interfaz simplificada exclusiva para los usuarios con rol 'client'. Tras iniciar sesión, el cliente ve únicamente sus mascotas y puede:

- Consultar el historial médico completo de cada mascota (solo lectura, sin poder modificar ni crear registros).
- Solicitar una nueva cita: seleccionar la mascota, el motivo, la fecha y una preferencia horaria. La cita se crea con estado 'pendiente' y aparece en el calendario del veterinario en color naranja para su confirmación.

Si el usuario cliente no tiene perfil vinculado (nuevo registro por Google OAuth), la aplicación le solicita completar sus datos personales y los de su mascota antes de acceder al portal.

6. Despliegue del Proyecto

El proyecto ha sido desplegado en un entorno de producción real en Hostinger, utilizando un servidor VPS con sistema operativo Linux y acceso SSH.

6.1 Servidor y Configuración del Entorno

El servidor de producción utiliza la siguiente infraestructura:

Componente	Detalle
Proveedor	Hostinger VPS
Sistema Operativo	Ubuntu 22.04 LTS
Servidor Web	Apache 2.4 con mod_rewrite
PHP	8.2 con extensiones necesarias
Gestor de Dependencias PHP	Composer 2.x
Base de Datos	MySQL 8.0 (Hostinger)
Frontend	Archivos estáticos compilados con Vite (dist/)
Protocolo	HTTPS con certificado SSL gratuito
Dominio	veterinaria.martinezyelamosabogados.es

Proceso de despliegue del backend (Laravel)

Los pasos realizados para desplegar el backend de Laravel en el servidor de producción son:

1. Subir el código fuente del directorio /back al servidor mediante SFTP o Git.
2. Instalar dependencias en modo producción: `composer install --no-dev --optimize-autoloader`
3. Copiar el archivo .env.example y configurar las variables de entorno de producción.
4. Generar la clave de aplicación: `php artisan key:generate`
5. Ejecutar las migraciones en la base de datos de producción: `php artisan migrate --force`
6. Ejecutar los seeders si es necesario: `php artisan db:seed --force`
7. Crear el enlace simbólico de almacenamiento: `php artisan storage:link`
8. Optimizar la aplicación: `php artisan config:cache && php artisan route:cache && php artisan view:cache`
9. Configurar el Virtual Host de Apache para que apunte al directorio /back/public.
10. Configurar el cron de recordatorios: añadir la tarea al crontab del servidor.

```
# Tarea cron para recordatorios automáticos (cada minuto, Laravel scheduler)
* * * * * php /ruta/al/proyecto/back/artisan schedule:run >> /dev/null 2>&1
```

Proceso de despliegue del frontend (React + Vite)

El frontend es una SPA estática compilada. Los pasos son:

11. Instalar dependencias: `npm install`
12. Compilar para producción: `npm run build`
13. Subir el contenido del directorio dist/ al servidor web.

14. Configurar el servidor Apache para que todas las rutas devuelvan index.html (necesario para el enrutador de React):

```
<IfModule mod_rewrite.c>
    RewriteEngine On
    RewriteBase /
    RewriteRule ^index\.html$ - [L]
    RewriteCond %{REQUEST_FILENAME} !-f
    RewriteCond %{REQUEST_FILENAME} !-d
    RewriteRule . /index.html [L]
</IfModule>
```

6.2 Base de Datos

La base de datos MySQL se encuentra alojada en el servidor de Hostinger. La configuración de producción en el archivo .env del backend es:

```
DB_CONNECTION=mysql
DB_HOST=localhost
DB_PORT=3306
DB_DATABASE=u233628092_veterinaria
DB_USERNAME=u233628092_veterinaria
DB_PASSWORD=[contraseña segura]
```

La conexión se establece mediante el socket local del servidor para máxima seguridad, sin exponer el puerto MySQL al exterior.

Las copias de seguridad se gestionan desde el panel de control de Hostinger, con respaldos automáticos diarios del sistema de archivos y la base de datos. Adicionalmente, se recomienda exportar periódicamente la base de datos con:

```
mysqldump -u usuario -p nombre_base_datos > backup_$(date +%Y%m%d).sql
```

6.3 URL del Proyecto

La aplicación web en producción está disponible en la siguiente URL pública:

<https://veterinaria.martinezyelamosabogados.es>

La API RESTful del backend está disponible en:

<https://veterinaria.martinezyelamosabogados.es/api>

Credenciales de acceso al entorno de demostración:

Rol	Email	Contraseña
Veterinario	admin@veterinario.com	password123
Asistente	asistente@veterinario.com	password123
Cliente	(email del propietario)	password123

El certificado SSL es gestionado automáticamente por Let's Encrypt a través del panel de Hostinger, renovándose de forma automática cada 90 días.